Unleash awesomeness. Private packages, team management tools, and powerful integrations.
Get started with npm Orgs »

jenkins-api

0.3.1 • Public • Published a year ago

Readme

1 Dependencies

20 Dependents

15 Versions

install

```
> npm i jenkins-api
```

weekly downloads

2,855

version

0.3.1

license

MIT

open issues

21

pull requests

4

homepage

github.com

repository

github

last publish

a year ago

collaborators

Test with RunKit

Report a vulnerability

nodejs-jenkins-api

npm v0.3.1 | downloads 8k/m | license package not found | issues 21 open

Install

```
npm install jenkins-api
```

Usage

Setup

```
var jenkinsapi = require('jenkins-api');

// no auth
var jenkins = jenkinsapi.init("http://jenkins.yoursite.com");

// username/password
var jenkins = jenkinsapi.init("http://username:password@jenkins.yoursite.com"

// API Token
var jenkins = jenkinsapi.init('https://username:token@jenkins.company.com');
```

If you need additional request parameters you can add them as explained in 'optional' section.

Builds

build

```
jenkins.build('job-in-jenkins', (optional){token: 'jenkins-token', ...}, func
  if (err){ return console.log(err); }
  console.log(data)
});
```

build_with_params

```
jenkins.build_with_params('job-in-jenkins', (optional){depth: 1, <param>:<val
  if (err){ return console.log(err); }
  console.log(data)
});
```

stop build

```
jenkins.stop_build('job-in-jenkins', 'build-number', (optional){token: 'jenki
  if (err){ return console.log(err); }
  console.log(data)
});
```

console output

```
jenkins.console_output('job-in-jenkins', 'buildname', (optional) {depth: 1, <
  if (err){ return console.log(err); }
  console.log(data)
});
```

build info

```
jenkins.build_info('job-in-jenkins', 'build-number', (optional) {depth: 1, <p
  if (err){ return console.log(err); }
  console.log(data)
});
```

last build info

```
jenkins.last_build_info('job-in-jenkins', (optional) {depth: 1, <param>:<valu
  if (err){ return console.log(err); }
  console.log(data)
});
```

last completed build info

```
jenkins.last_completed_build_info('job-in-jenkins', (optional) {depth: 1, <pa
  if (err){ return console.log(err); }
```

```
  console.log(data)
});
```

all builds

```
jenkins.all_builds('job-in-jenkins', (optional) {depth: 1, <param>:<value>, .
  if (err){ return console.log(err); }
  console.log(data)
});
```

test result/report

```
jenkins.test_result('job-in-jenkins', 'build-number', (optional) {depth: 1, <
  if (err){ return console.log(err); }
  console.log(data)
});
```

last build report - OBSOLET use `last_build_info`

```
// jenkins.last_build_report('job-in-jenkins', (optional) {depth: 1, <param>:
//   if (err){ return console.log(err); }
//   console.log(data)
// });
```

delete build data for job

```
jenkins.delete_build('job-in-jenkins', 'build-number', (optional) {depth: 1,
  if (err){ return console.log(err); }
  console.log(data)
});
```

Jobs

all jobs

```
jenkins.all_jobs((optional){token: 'jenkins-token', ...}, function(err, data)
  if (err){ return console.log(err); }
  console.log(data)
});
```

get config xml

```
jenkins.get_config_xml('job-in-jenkins', (optional) {depth: 1, <param>:<value
  if (err){ return console.log(err); }
  console.log(data)
});
```

update existing job configuration

```
jenkins.update_config('job-to-update'
                ,function(config) {
                    // function which takes the config.xml, and returns
                    // the new config xml for the new job
                    return config.replace('development','feature-branch');
                }
                ,(optional){token: 'jenkins-token', ...}
                ,function(err, data) {
                      // if no error, job was copied
                      if (err){ return console.log(err); }
                      console.log(data)
                });
```

update job

```
jenkins.update_job('job-to-update', xmlConfigString, (optional){token: 'jenki
  // if no error, job was copied
  if (err){ return console.log(err); }
  console.log(data)
});
```

job info

```
jenkins.job_info('job-in-jenkins', (optional) {depth: 1, <param>:<value>, ...
  if (err){ return console.log(err); }
  console.log(data)
});
```

create job

```
jenkins.create_job('job-in-jenkins', xmlConfigString, (optional) {depth: 1, <
  if (err){ return console.log(err); }
  console.log(data)
});
```

copy job

```
jenkins.copy_job('job-to-copy'
                ,'new-job-title'
                ,function(config) {
                    // function which takes the config.xml, and returns
                    // the new config xml for the new job
                    return config.replace('development','feature-branch');
                }
                ,(optional){token: 'jenkins-token', ...}
                ,function(err, data) {
```

```
            // if no error, job was copied
            if (err){ return console.log(err); }
            console.log(data)
        });
```

delete job

```
jenkins.delete_job('job-in-jenkins', (optional) {depth: 1, <param>:<value>, .
  if (err){ return console.log(err); }
  console.log(data)
});
```

enable job

```
jenkins.enable_job('job-in-jenkins', (optional) {depth: 1, <param>:<value>, .
  if (err){ return console.log(err); }
  console.log(data)
});
```

disable job

```
jenkins.disable_job('job-in-jenkins', (optional) {depth: 1, <param>:<value>,
  if (err){ return console.log(err); }
```

```
  console.log(data)
});
```

last success

```
jenkins.last_success('job-in-jenkins', (optional) {depth: 1, <param>:<value>,
  if (err){ return console.log(err); }
  console.log(data)
});
```

last result

```
jenkins.last_result('job-in-jenkins', (optional) {depth: 1, <param>:<value>,
  if (err){ return console.log(err); }
  console.log(data)
});
```

Queue

get all queued items

```
jenkins.queue((optional){token: 'jenkins-token', ...}, function(err, data) {
  if (err){ return console.log(err); }
```

```
  console.log(data)
});
```

get one queued item

```
jenkins.queue_item('queue-item-number', (optional) {depth: 1, <param>:<value>
  if (err){ return console.log(err); }
  console.log(data)
});
```

cancel queued item

```
jenkins.cancel_item('queue-item-number', (optional) {depth: 1, <param>:<value
  if (err){ return console.log(err); }
  console.log(data)
});
```

get all jenkins computers (aka workers)

```
jenkins.computers((optional){token: 'jenkins-token', ...}, function(err, data
  if (err){ return console.log(err); }
  console.log(data)
});
```

Views

get all views

```
jenkins.all_views((optional) {depth: 1, <param>:<value>, ...}, function(err,
      if (err){ return console.log(err); }
        console.log(data)
});
```

create view

```
jenkins.create_view('new-view-name', (optional)viewMode = 'hudson.model.ListV
  if (err){ return console.log(err); }
  console.log(data)
});
```

view info

```
jenkins.create_view('view-name', (optional){token: 'jenkins-token', ...}, fun
  if (err){ return console.log(err); }
  console.log(data)
});
```

update view

```
var viewConfig = {
            name: "view-in-jenkins",
            "description": "This is the view-in-jenkins View",
            "statusFilter": "",
            "job-in-jenkins": true,
            "useincluderegex": true,
            "includeRegex": "prefix.*",
            "columns": [{"stapler-class": "hudson.views.StatusColumn", "$clas
        };

jenkins.update_view('view-in-jenkins', viewConfig, (optional){token: 'jenkins
  if (err){ return console.log(err); }
  console.log(data)
});
```

delete view

```
jenkins.delete_view('view-in-jenkins', (optional){token: 'jenkins-token', ...
  if (err){ return console.log(err); }
  console.log(data)
});
```

add job to view

```
jenkins.add_job_to_view('view-in-jenkins', 'job-in-jenkins', (optional) {dept
      if (err){ return console.log(err); }
        console.log(data)
});
```

remove job from view

```
jenkins.remove_job_from_view('view-in-jenkins', 'job-in-jenkins', (optional)
      if (err){ return console.log(err); }
        console.log(data)
});
```

get all jobs in view

```
jenkins.all_jobs_in_view('view-in-jenkins', (optional) {depth: 1, <param>:<va
      if (err){ return console.log(err); }
        console.log(data)
});
```

Plugins

get all installed plugins

```
jenkins.all_installed_plugins((optional){token: 'jenkins-token', ...}, functi
    if (err){ return console.log(err); }
    console.log(data)
})
```

install a plugin

```
// var plugin = 'copyartifact@1.3.8';
var plugin = 'copyartifact@current';
jenkins.install_plugin(plugin, (optional){token: 'jenkins-token', ...}, funct
    if (err){ return console.log(err, data); }
    console.log(data)
});
```

NOTE: It will report successful even if the plugin is already installed. NOTE: Prevent Cross Site Request Forgery exploits need be disabled in Configure Global Security.

Default configuration

You can set the default configuration which will be use in all HTTP requests by calling init with the additional options parameter:

```
// default request options
var jenkins = jenkinsapi.init("http://jenkins.yoursite.com", {strictSSL: fals
```

Futhermore, you can set your remote job token for authentication:

```
// default request options
var jenkins = jenkinsapi.init("http://jenkins.yoursite.com", {strictSSL: fals
```

Since node-jenkins-api uses request/request as HTTP client, please refer to the documentation for available options.

Notes

Modeled after the Python Jenkins API

Keywords

github **jenkins**

You Need Help

Documentation
Support / Contact Us
Registry Status
Report Issues
npm Community Site
Security

About npm

About npm, Inc
Jobs
npm Weekly
Blog
Twitter
GitHub

Terms & Policies

Terms of Use
Code of Conduct
Package Name Disputes
Privacy Policy
Reporting Abuse
Other policies

npm loves you